Data Versioning and Reproducibility with DVC

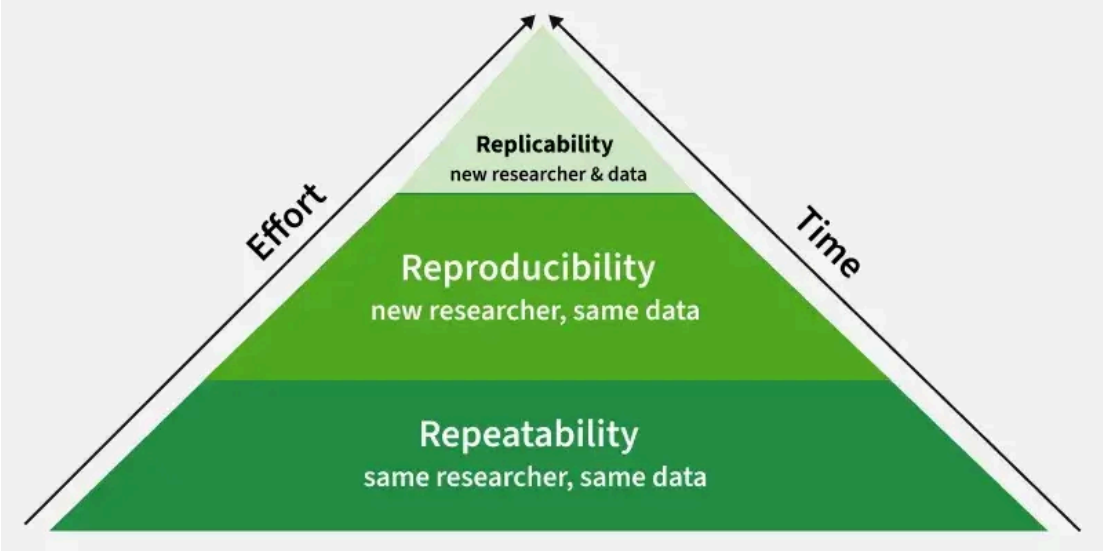
- **Course:** Engineering of Intelligent Models (MLOps & LLMOps)
- **Module:** M1. MLOps Foundations
- **Format:** Asynchronous Jupyter Notebook (self-study)
- **Focus Tool:** DVC (Data Version Control)
- **Version:** DVC 3.66+

1. Learning Outcomes

After studying this notebook, you should be able to:

1. Explain why traditional version control systems are insufficient for managing machine learning systems.
2. Describe the architectural principles behind Data Version Control (DVC).
3. Understand how DVC enables reproducibility, traceability, and collaboration in ML workflows.
4. Analyse the role of DVC within a broader MLOps architecture.
5. Conceptually integrate DVC into the Weather Forecast capstone system.

2. The Reproducibility Problem in Machine Learning

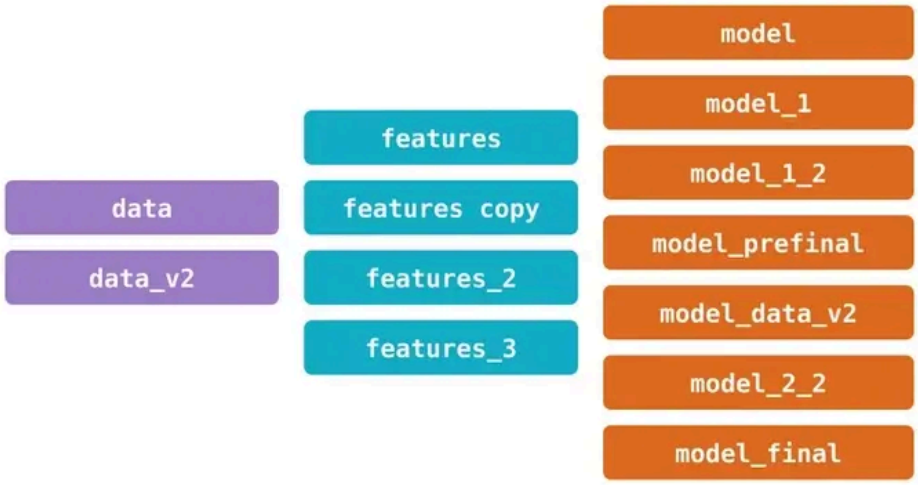


Reproducibility in machine learning means being able to run the same experiment again and get the same results. For ML this means using the same code, data and settings and making sure the model performs the same way each time. Reproducibility helps in debugging, comparing models, sharing work with others and deploying reliable systems in the real world. When experiments are reproducible teams can trust each other's work, improve on existing models, compare results fairly and avoid unexpected behavior during deployment.

In traditional software engineering, reproducibility is relatively straightforward. Source code is versioned, dependencies are declared, and builds can typically be reconstructed

deterministically from a repository state. Machine learning systems, however, introduce an additional and far more volatile dependency: **data**.

A trained model is not merely a function of its source code; it is a function of the dataset used during training, the preprocessing transformations, feature engineering logic, and hyperparameters. Without a tool like DVC, researchers often fall into the trap of manual versioning, leading to "filename hell."



In practice, datasets evolve continuously as records are appended, corrections are made, sensors drift, or APIs update. Without a structured way to track data versions, answering fundamental questions becomes impossible: Which dataset produced this model? Why did performance improve or degrade? Can we rebuild last month's model exactly as it was? In a production setting, these are operational, regulatory, and sometimes legal requirements. This is the problem that DVC was designed to solve.

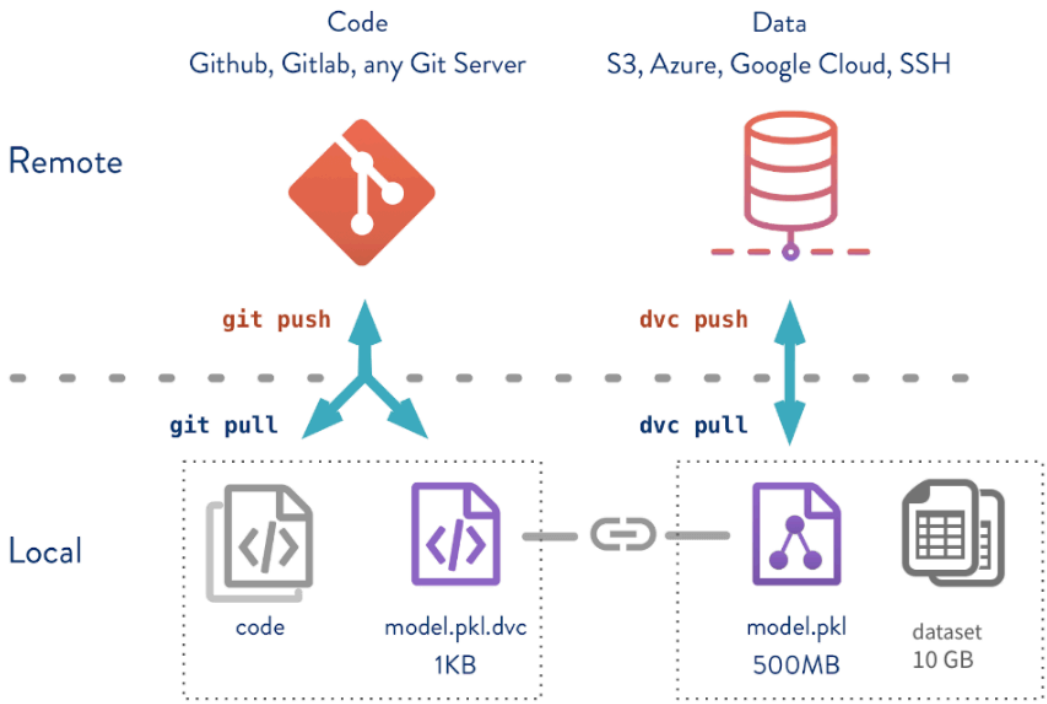
3. What is DVC?

DVC (Data Version Control) is an open-source system that extends Git-based workflows to large datasets and machine learning pipelines.

- **Official website**: https://dvc.org

- **Documentation**: https://dvc.org/doc

Unlike Git, which is optimised for tracking small text-based source code files, DVC is designed to manage large datasets, model artefacts, intermediate pipeline outputs, and reproducible ML workflows. DVC does not replace Git; instead, it integrates tightly with it. Git continues to manage source code and lightweight metadata, while DVC manages large files and pipeline state externally, linking everything to Git commits. This separation preserves the efficiency of Git while enabling data-centric version control.

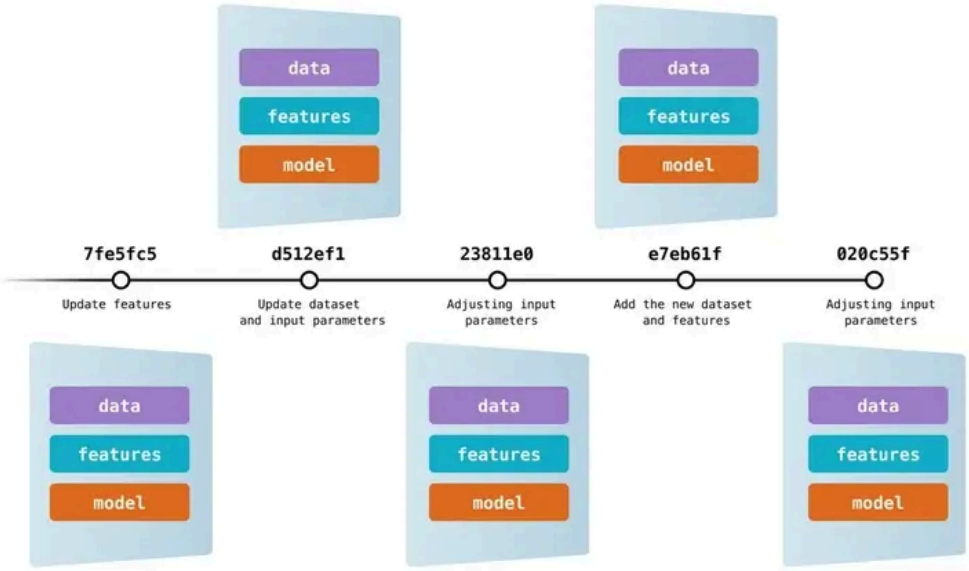


4. The Core Design Principle: The "Metafile"

The central idea behind DVC is to treat data and models as first-class citizens in version control. When you add data to DVC, it is not committed directly to Git. Instead:

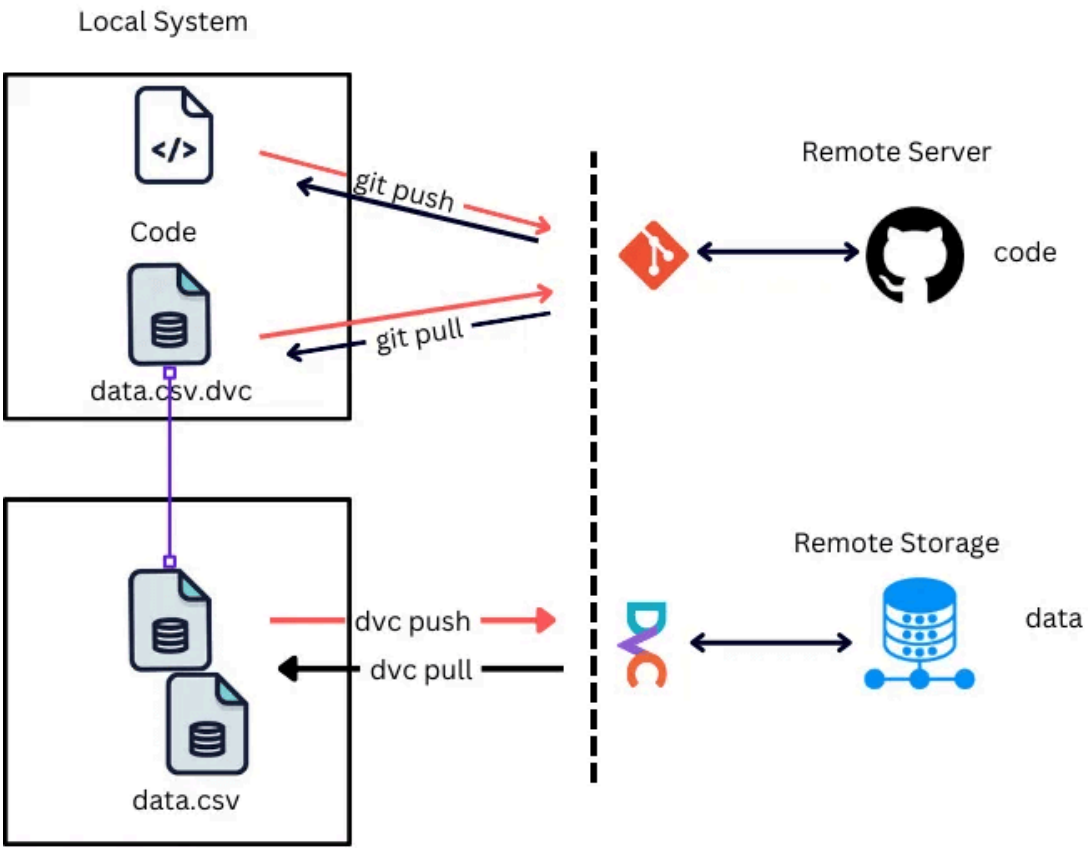
1. **DVC computes a cryptographic hash** of the file to uniquely identify its contents.
2. **The actual file is stored** in a dedicated storage area, known as the local cache or remote storage (like AWS S3 or Google Cloud Storage).
3. **A lightweight "Metafile" (`.dvc`) is created**. This file contains metadata and a pointer to where the actual data is stored.

Git tracks this `.dvc` metafile. Because the hash is content-based, any modification to the data produces a new version identifier in the metafile, creating a deterministic link between a Git commit, a specific dataset version, and a specific model artefact.



5. Remote Storage and Collaboration

Modern ML teams are often distributed, and data frequently exceeds gigabytes or terabytes. Storing such artefacts inside Git repositories is neither feasible nor efficient. DVC solves this through remote storage backends, which may include **Amazon S3**, **Google Cloud Storage**, **Azure Blob Storage**, SSH-accessible servers, or network storage systems. Team members can pull specific dataset versions, reproduce experiments locally, and share model artefacts without duplicating storage, supporting collaboration without sacrificing traceability, ensuring everyone works with the same data versions.



6. DVC within the MLOps Architecture of This Course

In our course stack, DVC serves as the structural base layer:

- **DVC** guarantees data and pipeline reproducibility.
- **Hydra** ensures configuration reproducibility.
- **MLflow** captures experiment metrics and metadata.
- **Airflow** orchestrates execution in time.
- **FastAPI/Docker** operationalise the model.

Without reproducible data and pipelines provided by DVC, tracking and orchestration lose their foundational meaning.

7. Recommended Further Reading

- **DVC Official Documentation**: https://dvc.org/doc
- **Made With ML – MLOps Guide**: https://madewithml.com
- **CRISP-ML(Q) Whitepaper**: https://arxiv.org/abs/2003.05155